

Ten Practices for Refactoring Scientific Pipelines to nf-core DSL2 on Shared Slurm HPC

Nil Tianchen Mu*, William Dizon*, Glen Otero*, Torey Battelle*

*Research Computing, Arizona State University, Tempe, AZ, USA

Email: {nilmu, wdizon, gotero3, tbattell}@asu.edu

Abstract—Long-lived scientific pipelines accumulate technical debt that is hard to remove by incremental patching: Python-orchestrated task generation, custom step or state managers, parameters mixed across scientific and infrastructure configurations, and monolithic job structures that bury per-task logs and error messages. We present ten generalizable practices for refactoring such pipelines into modern *nf-core DSL2* form on shared Slurm HPC clusters with Fairshare scheduling. Some restate the move to Nextflow and nf-core conventions: separating scientific from infrastructure configuration, and decomposing monolithic steps into containerized DSL2 processes. The more distinctive practices target execution under a contended shared scheduler: backfill-fit job sizing, `process.array` for many-task steps, walltime-based partition routing, Slurm controller-rate mitigations, and defensive failure handling that converts partial failures into easy-to-trace errors. The practices are distilled from refactoring `make_lastz_chains`, a bioinformatics pipeline for whole-genome pairwise alignment, and other pipelines on the Arizona State University supercomputers. The `make_lastz_chains` refactor was conducted with heavy use of AI coding assistants; we describe the engineering discipline that kept it safe and, in passing, surfaced hidden pre-existing bugs in the upstream pipeline. We further provide, to our knowledge, the first peer-reviewable quantitative measurement of Nextflow’s `process.array` directive on a production-scale scientific workflow under shared Fairshare-based Slurm HPC: three configurations across $N=3$ target–query sample pairs in a blocked-concurrent design on the public cluster, with a matched co-tenancy-free reference run per configuration on a single-user private cluster, comparing total walltime and Slurm scheduler stress. The `process.array` patch shortens walltime on the large shared cluster but not the small single-user one, while the full refactor is fastest on both. The refactored pipeline, container image, and parity-testing affordance are released as a public artifact structured for portability and reuse.

Index Terms—Workflow management systems, Nextflow, nf-core, HPC, Slurm, scientific software engineering, characterization testing, reproducibility, portability, software reusability, comparative genomics

I. INTRODUCTION

Long-lived scientific software pipelines accumulate technical debt at a rate that incremental maintenance cannot keep up with. In bioinformatics, a common pattern is a pipeline that began as a Python orchestration layer generating shell joblists for a workflow engine, evolved through a decade of feature requests and bug-fix patches, and now sits in a state where every new addition pays a structural cost. The ecosystem has, in parallel, converged on a modern alternative: *nf-core* [4], a community-curated framework of best practices and pipeline

templates built on Nextflow’s DSL2 syntax [6], and is now the de-facto reference style for bioinformatics workflows [5].

Migrating a legacy pipeline to nf-core is, however, not a mechanical exercise. The mismatch between nf-core’s cloud-native defaults and the realities of shared, Fairshare-scheduled Slurm HPC clusters introduces a class of issues that the nf-core literature does not address. Latent bugs in the original pipeline can survive a refactor unless the refactor is disciplined enough to surface them. And the modern reality is that significant fractions of refactoring work are now done with AI coding assistants, which both accelerate the typing and introduce new failure modes that the discipline must catch.

This paper presents ten generalizable practices for refactoring legacy bioinformatics tools to nf-core DSL2 on shared Slurm HPC. The practices are distilled from migrating `make_lastz_chains` [1], a whole-genome pairwise alignment pipeline whose original Python-orchestrated form we previously documented operating on the Arizona State University Phoenix cluster [2], [3], together with other pipelines refactored on the same clusters. We make four contributions: (1) ten practices organized by anti-pattern, recommendation, mechanism, and case-study evidence, four of which (P6–P8, P10) are not, to our knowledge, surfaced in the existing nf-core or Nextflow literature; (2) a three-part engineering discipline (pre-specification, characterization testing with tight output comparison, and full review of generated code) that kept the refactor safe under heavy AI assistance, scoped a fix for a previously reported upstream large-genome bug, and surfaced two further latent bugs in upstream; (3) a quantitative measurement of Nextflow’s `process.array` on a production-scale scientific workflow under shared Fairshare HPC (Section V), showing that a full nf-core refactor improves both walltime and scheduler health beyond what patching alone delivers; and (4) a reproducible Apptainer container, a parity-testing CLI affordance, and the refactored pipeline itself as a public artifact structured for portability and reuse.

II. BACKGROUND

A. *nf-core DSL2*

nf-core templates standardize pipeline directory layout, parameter schemas (`nextflow_schema.json`), container directives, conda environments, version capture (`versions.yml`), reporting, and linting [4], [5]. Pipelines that pass `nf-core lint` become eligible for inclusion in the nf-core pipeline registry, which provides automated

TABLE I: Sol vs. Phoenix Scheduler Parameters. Dev uses the same values as Phoenix.

Parameter	Sol	Phoenix
bf_max_job_assoc	25	2600
bf_max_job_user	50	—
bf_max_job_start	—	1300
bf_max_job_test	150,000	200,000
max_rpc_cnt	200	450
bf_yield_rpc_cnt	150	—
preempt_strict_order	—	set

continuous-integration testing and cross-platform reproducibility guarantees.

B. The `make_lastz_chains` pipeline

`make_lastz_chains` is a Nextflow pipeline for whole-genome pairwise alignment using LASTZ [8] and the UCSC Kent toolkit [1], [7]. It produces sorted alignment chains: gap-aware stitched alignments between two genomes. Downstream tools such as TOGA use these to infer orthologs (the same gene across species), and the NIH NCBI Comparative Genome Viewer renders them for browsing. A typical mammalian alignment fans out to ~40,000 LASTZ tasks; assemblies with many small scaffolds (contiguous assembly fragments shorter than a chromosome) can exceed 50,000 tasks per run. The original pipeline uses Nextflow as a parallel job runner orchestrated by a Python layer: `make_chains.py` generates joblist files that a Nextflow wrapper then executes. Our 2025 work [2] documented operational properties of this form on shared Slurm HPC: rapid Fairshare-score consumption, no per-task-job retry by default, and per-task exit codes not consistently propagated. Together with a year of upstream issue filings and a local deployment recipe [17], these produced a change list large enough to motivate a full refactor rather than continued patching. This paper reports that refactor.

C. ASU Sol, Phoenix, and Dev: cluster context and Slurm tuning

ASU operates three production HPC clusters relevant to this paper: Sol [18], [19], the flagship general-purpose supercomputer; Phoenix, the high-throughput companion tuned for many-small-task workloads, which hosts the benchmarks in this paper on its public CPU partition; and Dev, a small single-user private cluster that contributes a co-tenancy-free reference run per configuration to the benchmark in Section V, complementing the Phoenix measurements. All three run Slurm with Fairshare [9] and the backfill scheduler; the Sol/Phoenix Scheduler Parameters that matter for bulk submissions are in Table I (shared settings such as `bf_window=43200`, `bf_interval=30`, and `enable_user_top` are omitted).

Phoenix’s controller raises the per-association backfill cap by two orders of magnitude, more than doubles RPC concurrency, and adds an explicit per-cycle start ceiling (Table I); together these admit far more per-task jobs into each backfill cycle and absorb the RPC pressure a high-fan-out Nextflow workflow generates. Sol’s tighter caps trade that per-association throughput for global controller fairness. The

practices in Section III extrapolate to any Fairshare-plus-backfill Slurm site, but the absolute submission-rate ceilings shift with the controller tuning.

Terminology. We refer to the long-running Nextflow driver process as the main job, and to the individual Slurm jobs it submits per pipeline task as per-task jobs. We deliberately avoid the common label child job: at the Slurm level these jobs are independent submissions, not Slurm-tracked dependents, and using child implies a parent–child relationship the scheduler does not enforce.

D. Engineering discipline for refactoring

The methodological backbone of safe refactoring is the practice of characterization testing: capture the actual current behavior of the legacy code as a test, then change the code with the test as a safety net [10]. This idea has gained renewed importance as AI coding assistants are increasingly used for refactoring work; Beck has named the disciplined application of TDD-style practices with AI agents “augmented coding” [11], and Anthropic’s own guidance for the Claude Code agent recommends writing characterization tests as the first step of any legacy refactor [12]. Empirical work is more cautionary: a randomized controlled trial found AI tools slowed experienced open-source developers by approximately 19% even as they believed they were faster [13], and a study of LLM code generation identified “functional correctness hallucinations” (code that compiles and passes tests but does not satisfy the spec) as particularly hard to detect [14].

E. Literature gap: `process.array` benchmarking

Nextflow’s `process.array` directive batches per-task submissions into Slurm job arrays, reducing scheduler load. The directive has been documented as a stable feature since Nextflow v24.04.0 [15], but every public claim about its real-world impact comes from vendor blog posts [16] and HPC-site documentation; the peer-reviewed quantitative measurement is the gap Section V fills.

III. GENERALIZABLE PRACTICES FOR REFACTORING TO NF-CORE

We organize the practices as a numbered set, each opening with an *anti-pattern* lead followed by a *recommendation*. Practices P1–P5 and P9 are nf-core conventions and best-practice items, stated briefly. Practices P6–P8 and P10 are the differentiated contributions of this paper and are given in extended form, drawing where applicable on the additional elements *mechanism* and *case-study evidence* (P7 and P8 carry both; P6’s mechanism is inlined into its enumerated recommendation; P10’s evidence is folded into its recommendation through inline references to Bug 1). The anti-patterns are not specific to `make_lastz_chains`; we observe them in many bioinformatics tools that predate nf-core’s adoption.

P1: Decompose Python orchestration into native channels

Anti-pattern. A Python wrapper enumerates work items, writes them to a shell joblist, and invokes a workflow engine to

run the joblist; the engine sees one opaque per-stage invocation and cannot apply per-task caching, retry, or resume semantics.

Recommendation. Replace the joblist-file pattern with native Nextflow channel operators (`combine`, `flatMap`). Nextflow’s per-task caching, retry, and resume semantics then operate on individual tasks rather than on opaque joblist invocations. In the refactored `make_lastz_chains`, the LASTZ subworkflow uses `combine` to natively materialize the $N \times K$ target–query partition pairs, eliminating the joblist file and the Python wrapper from the parallel hot path.

P2: One process per tool; group into subworkflows by stage

Anti-pattern. A monolithic pipeline with no clean restart points relies on a custom JSON step-state manager (e.g. `--continue_from_step`) that becomes a source of confusion when users move intermediate outputs between directories.

Recommendation. Each module wraps exactly one tool and declares `label`, `conda`, `container`, `publishDir`, and a `versions.yml` emission. Modules group into subworkflows by pipeline stage. The result is independently testable units, version-tracked provenance, and clean restart points via Nextflow’s `-entry` flag. The refactor decomposes the original seven steps into 14 modules grouped under four subworkflows. Two named restart entries are exposed for the long-running stages of the pipeline: `FROM_FILL_CHAINS` resumes from a published `*.all.chain.gz` (via `--merged_chain`) and `FROM_CLEAN_CHAINS` resumes from a `*.filled.chain.gz` (via `--filled_chain`). Together they replace the legacy pipeline’s `--continue_from_step` flag and its custom JSON step-state manager with first-class Nextflow entry points.

P3: Backward-compatible legacy entry point

Anti-pattern. A refactor that replaces the original CLI forces every existing user to migrate on the refactor’s schedule and stalls adoption at sites where the legacy pipeline is embedded in established procedures.

Recommendation. Preserve the original CLI unchanged and add the new pipeline alongside it. Existing users see no change; new users opt in. This zero-disruption path materially lowers the cost of adopting a refactor at sites with established workflows. The refactor preserves `make_chains.py` verbatim; `nextflow run main.nf` is the new entry point.

P4: Separate scientific from infrastructure parameters

Anti-pattern. A single configuration file holds both scientific tuning parameters (e.g. LASTZ scoring) and infrastructure parameters (e.g. Slurm queue names); users editing one routinely break the other, and reproducing a run requires forking the config.

Recommendation. `params.json` holds versionable scientific parameters (LASTZ tuning, chain-cleaner thresholds); `nextflow.config` holds infrastructure (labels, profiles, container images). Reproducibility no longer requires forking

config. The separation also enabled the parameter audit (P9) by making it visible which parameters reach which layer.

P5: Containerize with dual conda+container directives

Anti-pattern. Tool installation by per-binary `wget` calls against version-mutable upstream sources is not reproducible: revisiting a run a year later finds different tool versions, and certain URL formats (e.g. GitHub `/blob/` pages) return an HTML wrapper rather than the raw source, producing build outputs that diverge from intent without a clear failure signal.

Recommendation. Each module declares both a `conda` environment and a `container` image; the active runtime profile (`-profile conda` or `-profile apptainer`) selects which is used. The refactor ships a single image (`nilablueshirt/make_lastz_chains:latest-amd64`) built reproducibly from a Dockerfile in the repo, holding the UCSC Kent toolkit, LASTZ, and pinned auxiliary scripts; `NXF_CONTAINER_IMAGE` overrides the default so sites without registry access can swap in a local build. The dual-directive plus override pattern makes the pipeline portable across Docker, Apptainer, and Singularity.

P6: HPC-aware overlay on nf-core defaults

Anti-pattern. `nf-core`’s standard `withLabel` resource tiers (`process_low`, `process_medium`, `process_high`) are designed primarily for cloud and on-premises environments without walltime-based partition routing. Applied unchanged to a Fairshare-scheduled Slurm cluster, they generate per-task `sbatch` submissions whose volume drops the user’s Fairshare score within minutes, push retries past partition walltime limits at submission, and route long jobs to short partitions or vice versa.

Recommendation. Layer six HPC-aware affordances over the `nf-core` defaults; the first is the load-bearing one and the others serve it.

- 1) **Backfill-fit per-task-job sizing.** A bulk submission on a Fairshare-scheduled cluster rapidly drops the user’s Fairshare score [3]; once it falls, Slurm’s main scheduler stops favoring the user’s per-task jobs and the workflow becomes dependent on the backfill scheduler to make further progress. Backfill dispatches a queued task only when it fits in a walltime gap between currently-running jobs: small jobs fit, large jobs wait. Right-sizing per-process resource and walltime requests is therefore not only an efficiency-hygiene measure but a scheduler-eligibility lever, distinct from the partition-cap clamping done by `check_max()` below. Size per-task-job requests by running a small representative test and reading the `nf-core report.html`, which records per-process peak CPU, memory, and duration for every task; `seff` reports only the main job’s allocation and is not sufficient. A smaller per-task-job profile also caps the main-job walltime the workflow needs: faster backfill dispatch means the main process completes its supervisory loop inside shorter allocations, which reduces the rate of

main-job timeouts (a failure mode that otherwise wastes the work of every per-task job that did complete).

- 2) **Slurm job arrays via `process.array`** [15]. For many-task steps, batch up to several thousand per-task jobs per `sbatch` submission, presenting a single job identifier to the scheduler at queue time. The refactor sets `array = 500` on LASTZ, AXT_CHAIN, and REPEAT_FILLER.
- 3) **Walltime-based partition routing.** Bind each `withLabel` tier to the partition whose walltime class fits. Short-running tasks (≤ 2 h) go to a higher-throughput partition; longer tasks go to general-purpose partitions. This eliminates a class of user error.
- 4) **Resource caps via `check_max()`.** Wrap dynamically scaled resource requests (`memory = { 4.GB * task.attempt }`) with a helper that caps each scaled request against partition hard limits. Retried jobs no longer get rejected at submission for asking too much.
- 5) **Slurm controller and submission-rate mitigations.** Long high-volume runs can outpace the Slurm controller’s RPC capacity, producing ghost tasks (NODE_FAIL with no `.exitcode` file) and `slurm_load_jobs` socket timeouts. Five executor-block knobs reduce the pressure without changing pipeline throughput: `pollInterval 5s→30s` (slows the JVM tick), `queueStatInterval = '2 min'` (halves `squeue` polling), `exitReadTimeout = '15 min'` (waits for `.exitcode` so wedged tasks fail cleanly and `errorStrategy='retry'` can fire), `queueSize` (caps in-flight tasks against the controller), and `killBatchSize` (batches `scancel` on shutdown). For development iteration, scale `queueSize` and `array` down so a small parity input completes a feedback loop in minutes.
- 6) **Pipeline-level data-prep cache for many-task steps.** When all tasks in a many-task subworkflow consume slices of the same shared input, pre-extract once at the prepare-genomes stage and stage the cached directory into every downstream task as a path input; Nextflow symlinks the directory into each task work dir at zero marginal storage cost. The refactor’s `EXTRACT_CHROMS` module materializes per-chromosome FASTAs once per genome for the V1 .2bit reader path (Section IV-C); re-extracting inside every LASTZ task instead would multiply scratch storage by the LASTZ task count (~ 50 k for a production run) and cause BULK-partition tasks spanning many small scaffolds to exhaust their walltime on redundant extraction. A per-task fallback (`./_v1_chrom_cache/` in the task work dir) preserves correctness for legacy entry points that bypass `EXTRACT_CHROMS`.

These knobs are intentionally observable: `nf-core's default report.enabled, timeline.enabled, and trace.enabled` produce per-task resource and timing arti-

facts that tell the operator whether each affordance is hitting in production, in particular whether per-task jobs are landing in backfill windows or sitting in the main-scheduler queue.

Evidence. Section V measures the job-array overlay on walltime and scheduler stress; the Fairshare $\rightarrow$ backfill chain and its Fairshare-score trajectory under a bulk submission are documented quantitatively in our companion operator-audience presentation [3].

P7: Strict-by-default error handling and post-stage integrity checks

Anti-pattern. `errorStrategy = 'retry'` combined with `maxErrors = '-1'` silently tolerates an unbounded number of permanently failed tasks. A task that exhausts its retries is marked FAILED but the workflow continues without its output. If the failure mode also happens to produce no error message (as in the silent-failure constructions of P10), the run completes “successfully” with missing data.

Recommendation. Four layers of defense.

(i) **Strict-by-default `errorStrategy`.**

Listing 1: Strict-by-default `errorStrategy`.

```
errorStrategy = { task.attempt <= 8
                  ? 'retry'
                  : 'terminate' }
maxRetries     = 8
```

(ii) **Post-stage integrity-check assertion.** At the end of any many-task stage, materialize the expected ($target \times query$) pair list and compare its length against the count of successful task completions. If they diverge, abort before downstream stages read incomplete data. Implemented in `subworkflows/local/lastz_alignment/main.nf`:

Listing 2: Post-stage integrity-check pattern.

```
def expected_n = pair_list.count()
def actual_n   = LASTZ.out.versions.count()
expected_n
  .combine(actual_n)
  .subscribe { exp, act ->
    if (exp != act) {
      error "LASTZ_integrity_check_failed:_" +
        "expected_${exp}_alignment_tasks,_"
      +
        "only_${act}_produced_output."
    }
  }
```

(iii) **Out-of-band watchdog.** The two preceding layers protect against task-level failure modes inside Nextflow. The residual failure mode is a wedged main job that has lost sight of a task that disappeared from Slurm without writing `.exitcode` (a NODE_FAIL ghost or an RPC-timeout drop). Because no failure ever reaches Nextflow, `errorStrategy='retry'` cannot fire and the run hangs indefinitely. A small external watchdog (running on the login node, in `tmux/screen`, or as a separate small Slurm job) detects this case by content-hashing the most recent “tasks to be completed: N” block in `.nextflow.log`. If the hash is unchanged for `STALL_THRESHOLD` seconds

(default 30 min), the watchdog `scancel`s the main job and resubmits it with `-resume`. A `MAX_RESTARTS` cap (default 5) prevents infinite resubmit loops on a sick cluster. The watchdog never runs as part of the main job itself, since the `scancel` it issues would then kill the watchdog along with the main job.

(iv) Graceful per-task-job termination on the wall-time signal. The three preceding layers detect failures after they happen. The residual failure mode is a per-task job that reaches its Slurm walltime hard limit mid-write: Slurm sends `SIGTERM` immediately followed by `SIGKILL`, the partial output file may still satisfy an `optional: true` declaration, and the task is recorded as having “produced” truncated output. Pre-empt this by adding `clusterOptions = '--signal=B:USR1@60'` to many-task processes and trapping the signal in the task script to remove partial outputs and exit non-zero before Slurm’s hard kill arrives. The task then surfaces as a clean failure that `errorStrategy='retry'` and the integrity check (ii) can act on, converting a hard-kill silent-truncation pathway into the same explicit-failure path as any other task error.

Mechanism. Each layer closes the residual gap left by the ones above it, and together they reject the outcome class in which downstream chain building runs against incomplete LASTZ output.

Evidence. The first two layers would have caught Bug 1 (Section IV-B) at the first run; the watchdog recovered a ~40k-task amphibian run that wedged on a Slurm RPC storm in production.

P8: Parity-testing affordances for divergent code paths

Anti-pattern. A pipeline that branches behavior by input size, file format, or version exercises the rare branch only on rare inputs. Bugs in the rare branch surface for the first time on the first slow run that triggers it.

Recommendation. For any code-path divergence introduced for an edge case, add a CLI flag that forces the rare path on inputs small enough for tight feedback loops, and pair it with a small comparator that quantifies any divergence between the two paths’ final outputs (overlap matching on the alignment-bucket key rather than exact equality, since the rare branch may legitimately drift on shared upstream tool internals). Mark the flag as a debug affordance so it does not appear in `params.json` (which should remain reserved for scientific parameters) but is recognized by the schema validator.

The refactored pipeline supports two `.2bit` format versions for processing the input files: V0 (32-bit offsets, the default and the version LASTZ reads natively) and V1 (64-bit offsets, required for genomes over 4GB but unreadable by LASTZ directly). The reader path branches on detected version. To exercise the V1 extraction path on small genomes, we added a CLI flag that forces the rare `.2bit` V1 code path on small inputs, plus a chain comparator that reports matched/missing chains, score distribution against baseline, and target-partition-boundary proximity:

Listing 3: Parity-testing flag and chain comparator.

```
nextflow run main.nf -params-file params.json \
--outdir results_v1 --force_long_2bit
python3 standalone_scripts/compare_chains.py \
results_v0/final/*.final.chain.gz \
results_v1/final/*.final.chain.gz
```

Mechanism. The flag converts a code path that would otherwise be exercised only on ~40 GB genomes into one that runs against chicken-vs-zebra-finch in minutes. Bugs in the rare branch get the same feedback loop as bugs in the common branch.

Evidence. The flag caught Bug 3 (Section IV-C) on a small genome before any production-scale run. The flag plus comparator also enables a standing quantitative parity check between the two reader paths (Section IV-B), which on the tailed frog vs. chicken reference reproduces upstream chain-for-chain on V0 and isolates the V1 drift to ~0.11 % of chains in the low-score tail, away from target-partition boundaries. The drift is therefore attributable to lastz-internal soft-masking differences between the `.2bit` index and a FASTA stream, not to pipeline orchestration.

P9: Audit parameters during migration

Anti-pattern. Long-lived pipelines can accumulate CLI flags that no tool invocation consumes; users may set them without observable effect.

Recommendation. Cross-check every CLI parameter accepted by the legacy interface against actual tool invocations across every stage. The migration is the natural moment to surface and remove dead flags. The audit found three dead parameters in upstream (`--seq1_limit`, `--seq2_limit`, `--fill_chain_min_score`); the refactored pipeline omits them.

P10: Recognize the silent-failure construction kit

Anti-pattern. The pattern `subprocess.call(...)-without-return-code-check`, combined with workflow-level `optional: true` outputs, is a silent-failure construction kit. Either layer alone surfaces failures: a wrapped tool that exits non-zero raises in Python if the wrapper checks the return code; a missing required output raises in Nextflow if the output is not optional. Together they swallow failure completely. The wrapped tool exits non-zero, `subprocess.call` returns the exit code, the wrapper ignores it and exits zero itself, Nextflow records task success, and the absence of the expected file is permitted by `optional: true`. The run completes with missing data and no error trail.

Observability dimension. A Python orchestrator dispatching all per-tool work inside a single Nextflow job emits one `stdout/stderr` pair and a flat `.nextflow.log`, so a swallowed failure leaves no per-task trail. P2’s one-process-per-tool structure restores the layered observability Nextflow and Slurm provide by default: each task work dir carries its own `.command.{sh,out,err}` and `.exitcode`, and `nf-core’s trace.txt/report.html/timeline.html` give the cross-task view. A P10-shaped bug then surfaces as a

TABLE II: Ten-practice refactoring checklist. Bold numbers (P6–P8, P10) are this paper’s differentiated contributions, not previously surfaced in the nf-core or Nextflow literature.

#	Practice
P1	Decompose Python orchestration into native channels.
P2	One process per tool; group into subworkflows by stage.
P3	Backward-compatible legacy entry point.
P4	Separate scientific from infrastructure parameters.
P5	Containerize with dual conda+container directives.
P6	HPC-aware overlay on nf-core defaults: backfill-fit per-task-job sizing, <code>process.array</code> , walltime partition routing, <code>check_max()</code> , Slurm controller / submission-rate mitigations, and a pipeline-level data-prep cache for many-task steps.
P7	Strict-by-default <code>errorStrategy</code> , post-stage integrity-check assertion, an out-of-band watchdog for wedged main jobs, and walltime-signal trapping for graceful per-task-job termination.
P8	Parity-testing CLI affordance for any divergent code path.
P9	Audit CLI parameters during migration; remove dead flags.
P10	Recognize and refuse the <code>subprocess.call + optional: true</code> silent-failure construction kit.

missing artifact in a known work dir rather than vanishing into a monolithic log.

Recommendation. Always check return codes when wrapping shell calls. Reserve `optional: true` for outputs that are genuinely optional in the science (a stage that emits a side-channel report only under specific conditions), not for “the tool sometimes fails to write this.” This anti-pattern produces what the LLM-code-generation literature calls a functional-correctness hallucination [14] when found in generated code; we observe it in non-generated legacy code as well (Bug 1, Section IV-B).

Synthesis: a refactoring checklist

Table II summarizes the ten practices in checklist form for reuse. P1–P5 are nf-core conventions; P6–P8 and P10 are the differentiated contributions of this paper.

IV. ENGINEERING DISCIPLINE: PRE-SPEC, PARITY, REVIEW

The practices in Section III describe what a refactored nf-core pipeline should look like. This section describes how we conducted the refactor itself in a way that surfaced latent bugs and remained safe under heavy AI assistance. The discipline has three parts: pre-specification from observation, characterization testing with tight output comparison, and full review of generated code. It is a specialization of long-standing practice [10], [11] to scientific-software refactoring on shared HPC. Each part of the discipline maps to one of three production bugs in upstream: Bug 2 was filed against the upstream repository by the Earth BioGenome Project before this refactor began (Section IV-A) and the pre-spec scopes its fix into the refactor rather than letting it slip; Bugs 1 and 3

were newly surfaced by the parity check (Section IV-B) and the review step (Section IV-C) respectively.

The refactor was conducted with primary use of the Claude Code agent and with the Qwen3 model used in parallel for cross-checking alternative implementations. We do not study the AI tooling itself; we describe the discipline that made its use safe.

A. Pre-specification from Observation

Before any code generation, we built and recorded a change list from direct observation of the original pipeline running on real data. The record is durable and predates the refactor: it consists of our 2025 operational paper [2], public bug and improvement requests filed under NilaBlueshirt on `hillerlab/make_lastz_chains`, and an operational deployment recipe in a separate repository [17]. Pre-spec artifacts that exist before the refactor cannot be retrofitted; they make the discipline verifiable.

Bug 2 evidence. Bug 2 is not a bug we surfaced. It was filed by collaborators in the Earth BioGenome Project as Issue #56 [20] on the upstream repository, recording that `make_lastz_chains` failed at the FASTA-to-.2bit conversion step (`faToTwoBit`, where FASTA is the standard text format for DNA sequences and .2bit is a UCSC compact binary form) on large vertebrate genomes (lungfish ~40 GB, salamander ~21 GB) with the error “index overflow — use `-long` option,” and that adding `-long` caused `faToTwoBit` to succeed but LASTZ subsequently failed to read the resulting .2bit file. The discipline-relevant claim here is not the bug’s discovery but the pre-spec’s role in routing the fix: the issue thread captured the symptom before the refactor began, the pre-spec listed it as in-scope, and the refactor’s mechanism for fixing it (size-aware writer + version-aware reader) is a contribution of this paper. Without the pre-spec record, the size threshold and the dual code paths would have been an after-the-fact patch indistinguishable from drift.

B. Characterization Testing and Tight Output Comparison

Capture upstream’s reference output on a small but realistic input, then gate the refactored pipeline on byte-equivalence (or sorted-equivalence, where the tool’s output ordering is non-deterministic). For `make_lastz_chains` the parity check compares the final `*.all.chain.gz` after sorting.

Bug 1 evidence. Validating the refactored pipeline against upstream’s published outputs on a chicken vs. zebra finch reference run revealed a substantially smaller final `*.all.chain.gz` than upstream’s. Per-target-chromosome counts showed chains for thousands of small scaffolds were missing while chains for the few large chromosomes were present, and every LASTZ task had been reported successful with no error emitted. The fraction of alignment coverage lost is not constant across samples; it scales with the number and size distribution of small scaffolds. We did not quantify the loss across every run, but in one amphibian sample we examined it reached ~23% of alignment coverage, illustrating how large a silently dropped fraction can be.

The trigger was a filename-length overflow in upstream’s `run_lastz_intermediate_layer.py`: BULK partition strings (which list up to 100 scaffold names) produced per-task alignment output filenames (`*.psl`, LASTZ’s pairwise alignment format) longer than the 255-byte filesystem limit, and the resulting non-zero exit was masked by exactly the P10 anti-pattern in the wrapper. Every BULK partition’s alignments were absent from the final output. The bug is latent in upstream’s main branch and affects any genome with many small scaffolds; without an external comparison against upstream’s reference output the loss would have propagated to downstream users undetected. The full per-target-chromosome chain-count comparison that surfaced the loss is recorded in the refactor pull request [21]. Surfacing this bug was not the aim of the refactor. The aim was a maintainable pipeline; P2’s one-process-per-tool structure, the strict error handling of P7, and the return-code discipline of P10 are what convert a loss like this from silent missing data into an explicit failure, so a comparable silent drop cannot recur undetected in the refactored pipeline.

Positive parity result. Once Bug 1 was fixed, the same discipline, wired into the chain-level comparator at `standalone_scripts/compare_chains.py`, returned a positive result on *tailed frog* vs. *chicken*: the V0 path reproduces upstream chain-for-chain (643,861 chains on both sides), and the V1 path forced via `--force_long_2bit` shows only $\sim 0.11\%$ truly-missing chains concentrated in the low-score tail (0 of 714 within $\pm 10\text{kb}$ of any target-partition boundary). This is a signature consistent with *lastz*-internal differences between *.2bit*-indexed and FASTA-stream input, not pipeline orchestration, and in regions downstream tools discard. The positive result is the companion of the Bug 1 finding: the same discipline that catches silent failure also calibrates how much of any residual divergence is ours to fix versus inherited from tool internals. Full counts are in the refactor pull request [21].

C. Full Review of Generated Code

Read every line of generated code rather than trusting it. The pre-spec constrains what the refactor should be doing; the parity check verifies the end-to-end output; the review catches the residual class of plausible- looking but semantically wrong code. The two-model setup (Claude Code as primary, Qwen3 as cross-check) is a robustness affordance for the review step: divergent suggestions are the cheapest way to surface assumptions either model is making implicitly.

Bug 3 evidence. The first execution of the V1 *.2bit* code path under `--force_long_2bit` (the parity affordance from P8) crashed every LASTZ task with `FAILURE: fopen_or_die failed to open ".../<file>.fa/<chrom>".` The cause: LASTZ’s documented `<file>/<seqname>` input syntax is implemented only for *.2bit* inputs; for FASTA inputs LASTZ takes the string literally and tries to open it as a filesystem path. The V1 fix had introduced FASTA dispatch but left the `seqname` selector in the command builder. The

error was plausible-looking generated code: the syntax pattern is correct for *.2bit* and the code did the “obvious” thing for FASTA. Code review caught it within the small-input feedback loop the parity flag enabled. Without the affordance, the bug would have surfaced for the first time on the next $\sim 40\text{GB}$ lungfish or salamander run.

D. Why the Discipline Matters More under AI Assistance

The discipline is not specific to AI-assisted refactoring, but empirical evidence [13], [14] suggests it matters more as AI assistance grows. The P10 anti-pattern is the same class Liu et al. [14] name; AI assistance makes it more frequent but does not invent it. Bug 1 occurred in non-AI-generated legacy code.

V. PERFORMANCE EVIDENCE

A. Experimental setup

We compared three configurations on the Phoenix public partition and on the Dev single-user cluster (Section II-C):

- 1) **Original.** Upstream `make_lastz_chains` (Python-orchestrated joblists, no job array, no container).
- 2) **Original + Job Array.** The upstream pipeline with `process.array` enabled and the best-practice configuration of our prior work [2].
- 3) **nf-core refactor.** The DSL2 refactor described in this paper, run with `-profile apptainer, slurm`.

The three configurations form a deliberate progression of pipeline-modernization strategies rather than a single-variable ablation, so the comparison evaluates the refactor as a whole. All three were run with identical Nextflow executor parameters (`pollInterval`, `queueSize`, `submitRateLimit`, `exitReadTimeout`, `killBatchSize`) set to the controller-friendly values from P6, so executor tuning is removed as a confound and any remaining differences are between the three strategies.

Dev and Phoenix use the same Slurm Scheduler Parameters (Table I), and all benchmark runs use the same scientific parameters and the same Nextflow configuration parameters. The only differences are the three modernization strategies under test and each cluster’s partition routing.

Phoenix’s public CPU partition (production conditions) was exercised with $N=3$ independent sample-pair blocks in a blocked-concurrent design: each block uses a distinct target-query sample pair, and within a block all three configurations were submitted concurrently with start order randomized across blocks. The three sample pairs are the three independent experiments, giving $N=3$ measurements per configuration with sample-pair variation treated as the noise term. The design reflects production usage: a single workflow cannot saturate the partition under the controller’s RPC pressure budget (P6), so 3–5 concurrent workflows is typical. Within a block the three configurations contend both with each other and with other tenants, exactly as a user running several concurrent workflows would experience. Each block still controls for time-varying background load, and the three sample pairs spread biological-input variance across the run. The Dev

single-user cluster was exercised on the same three configurations with one sequential reference run each, on a sample pair comparable to the Phoenix set. Each Dev run is windowed to the LASTZ stage, from the first LASTZ submission to the last LASTZ completion: LASTZ is the bulk submission wave that collapses Fairshare and concentrates the controller stress this section measures, so it is the most informative window for the scheduler-stress comparison, whereas Phoenix is measured over the full pipeline under production conditions. A single run per configuration is sufficient by construction rather than a budget compromise: with no co-tenants and per-user Fairshare reset before each run, a same-input repeat is deterministic to the resolution of every quantity we report. The repeat reproduced the LASTZ-stage walltime within $\sim 1.5\%$ and the submission count exactly. Dev’s contribution is attribution, not variance: cluster-global controller activity is directly attributable to the workflow under test, giving an absolute-value LASTZ-stage reference the Phoenix blocked-concurrent design cannot, on the same three-configuration axis.

All three sample pairs are below the 4 GB threshold, so every run takes the V0 .2bit path (P8); a sanity-check run of tailed frog vs. chicken with `--force_long_2bit` showed total walltime within one hour of the matching V0 run, below single-block variance. The V1 path’s added steps (P6 item 6 EXTRACT_CHROMS plus FASTA dispatch) therefore do not affect the configuration ordering reported here. Per-run we collected `.nextflow.log`, Nextflow’s `trace.txt` (per-task submission, start, completion, exit status, and allocated resources), the final `*.all.chain.gz` (with per-target chain counts as a correctness gate), and a `meta.json` (configuration, block, order-in-block, Nextflow version, container digest, main-job ID, starting Fairshare). We do not use `sacct` as a per-task data source: at the per-run task counts involved (tens of thousands), `sacct` queries are prohibitively slow, and the same fields are already emitted by `trace.txt` at zero controller cost. Per Phoenix block, `sdiag` was sampled every 300s for the duration of the measurement window, starting ~ 5 min before the workload began so the first inter-sample delta covers the workload onset. No `sdiag --reset` was issued on Phoenix (it requires `SlurmUser` privilege and adds controller RPC pressure on a shared partition); instead, all rate metrics in Fig. 4 are computed as inter-sample deltas of `sdiag`’s cumulative counters, making them independent of the reset-window state. On Dev, where we hold `SlurmUser`, per-user Fairshare is reset before each run so every configuration starts under an identical regime; we issue `sdiag --reset` between runs as hygiene, but the same inter-sample-delta computation is applied, so the reported metrics are independent of the reset-window state on both clusters.

a) *Failure handling and walltime cap.*: Configurations that do not complete unattended are reported with walltime-to-failure (the headline number, reflecting an unsupervised production run) and supervised-completion walltime (walltime-to-failure plus a single manual rerun of the failing stage), with the failing stage and its resource-exhaustion mode noted in the text. We did not retune failing configurations, since the failure

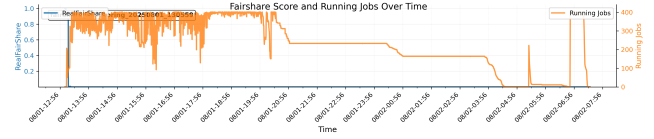


Fig. 1: Fairshare-score trajectory under a bulk submission, establishing the backfill-dependency premise for this study. Reproduced from prior work [3] as motivating context, not a result of the present benchmark.

mode itself is a finding (Section V-C). A pre-specified 24 h walltime cap is applied uniformly across configurations and blocks; runs exceeding the cap are right-censored, which does not affect the headline ordering between configurations.

B. Fairshare and backfill dependency

Per Fig. 1, the user’s Fairshare score drops to near zero within minutes of the first wave of LASTZ per-task-job submissions and stays there for the rest of the run. Once Fairshare falls, Slurm’s main scheduler stops favoring the user’s per-task jobs and further progress depends on the backfill scheduler. This is the prerequisite mechanism that motivates the backfill-fit per-task-job sizing in P6, and the backfill regime under which the walltime and scheduler-stress measurements below characterize each configuration.

C. Walltime

Per-configuration walltime is read off the curves in Figs. 2 and 3: on Phoenix each bold curve is annotated at its endpoint with the average walltime across the $N=3$ blocks for that configuration; on Dev each curve is the single reference run, annotated with its LASTZ-stage walltime. The configuration ordering is Original $\gg$ Original+Job Array $>$ nf-core refactor on Phoenix public: the Original $\rightarrow$ Job Array step is the largest single drop and isolates the contribution of `process.array` alone, while the remaining Job Array $\rightarrow$ nf-core gap reflects the full DSL2 refactor as a whole (native channel parallelism, container-cached tool startup, and finer process decomposition). On Dev the ordering differs, and the difference is itself a finding: the refactor is again fastest, but Job Array runs slower than Original (reproduced in the repeat, so not measurement noise), so the patch’s submission-batching advantage does not survive the change in cluster scale. The refactor wins on both clusters; the cluster-scale dependence of that benefit is analyzed in our companion operator-audience presentation [3]. Censored runs (walltime ≥ 24 h) carry the supervised-completion walltime (Section V-D); the cap is a right-censoring point and does not change the ordering.

D. Scheduler stress

The primary metric is the per-pipeline `sbatch` submission rate, derived from each run’s `.nextflow.log` by counting unique array- base Slurm IDs (all tasks of one `sbatch --array=...` call share a base) and cross-checked against Nextflow’s `trace.txt`.

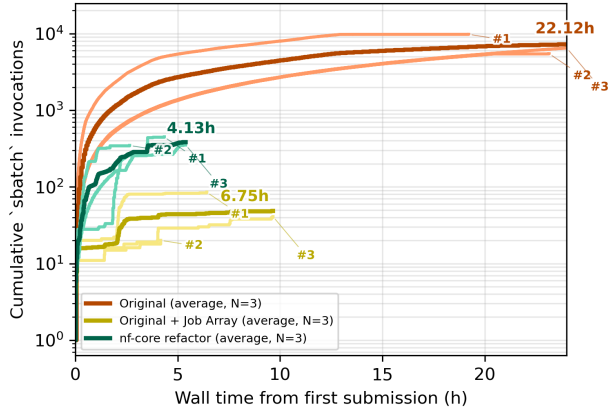


Fig. 2: Cumulative distinct `sbatch` submissions vs. walltime on Phoenix public over the full pipeline ($N=3$ blocked-concurrent blocks per configuration). Thin curves are individual blocks (labeled #1/#2/#3 just beneath curve end); bold curves are the per-configuration average, labeled with the average walltime (h). All curves are clipped to the uniform 24 h walltime cap.

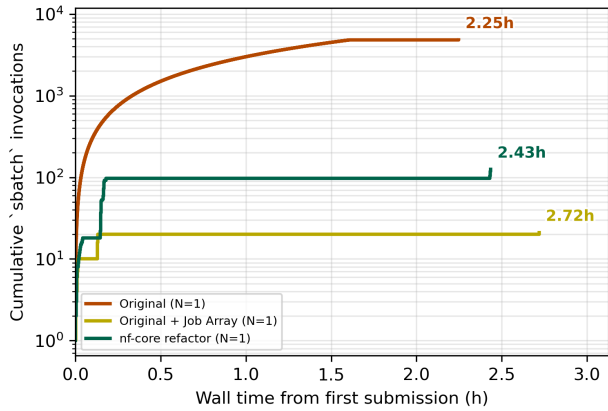


Fig. 3: Cumulative distinct `sbatch` submissions vs. walltime on the Dev single-user cluster, over the LASTZ stage (one co-tenancy-free reference run per configuration, on a sample pair comparable to the Phoenix set). Each curve is labeled at its endpoint with its LASTZ-stage walltime (h). All curves are clipped to the uniform 24 h walltime cap.

Figs. 2 and 3 report the per-pipeline submission rate on Phoenix public (full pipeline) and Dev (LASTZ stage, the dominant submission wave). Original submits one `sbatch` per LASTZ task; both array-enabled configurations batch up to `array=500` per submission. The refactor sits modestly above Original + Job Array because its finer process decomposition (P2) emits more array-base submissions, still orders of magnitude below Original, and those extra submissions buy the testability and recovery semantics of Section III. Both clusters agree on this submission-count ordering, which is structural; on Dev, however, walltime does not track it, as Job Array issues far fewer submissions than Original yet runs slower (Section V-C).

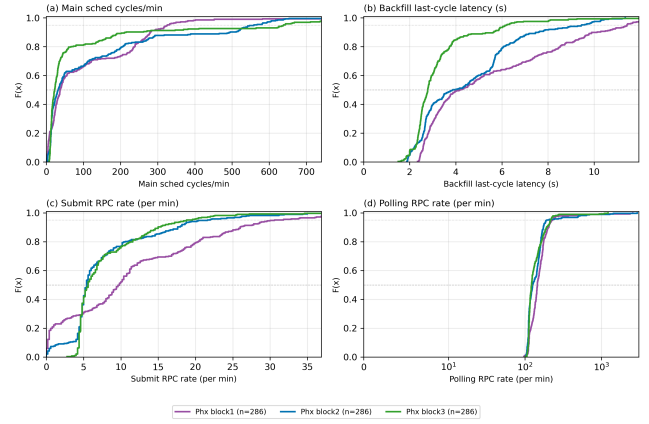


Fig. 4: Empirical CDFs of cluster-global Slurm controller metrics on Phoenix public.

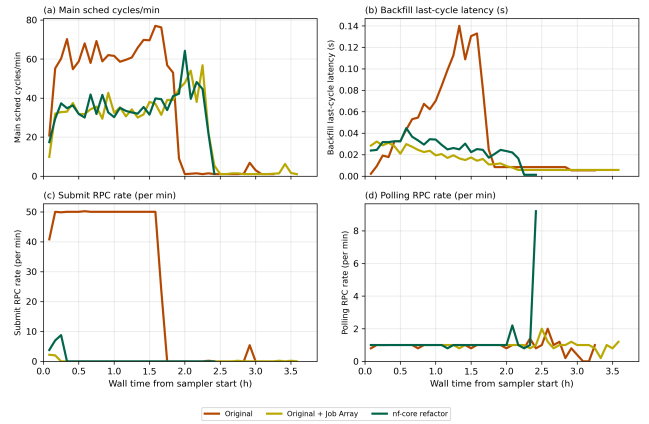


Fig. 5: Cluster-global Slurm controller metrics on Dev over the LASTZ stage, by configuration. The late spike on the refactor curve in panel (d) is a measurement artifact: the refactor pipeline does not report LASTZ-stage completion in its job log the way Original and Original + Job Array do, so we identified the stage end with manual `squeue` polls, and those checks themselves register as polling RPCs.

Fig. 4 provides the controller-side view on Phoenix across four cluster-global metrics: main-scheduler throughput (a), backfill last-cycle latency (b), submit-RPC rate (c), and per-task polling-RPC rate (d). Each panel shows one empirical CDF per measurement block over the first 24 h of that block’s sampler window. Overlap among the three blocks’ CDFs is the co-tenancy control: closely overlapping curves indicate each block sampled a comparable distribution of ambient controller state, so configuration ordering observed within a block is not an artifact of one block running on a busier or quieter day. A distributional view is correct here because the three blocks ran on different real days, so temporal alignment is not defensible but distributional similarity is. Cumulative counters (panels a, c, d) are differentiated between consecutive samples before the ECDF is estimated (Section V-A). Panel (d) isolates the controller-side cost of per-task `squeue/scontrol` traffic that `process.array` eliminates as a side-effect of submission batching, the distributional view of the status-polling load

flagged as unmeasured in Section II.

Fig. 5 provides the matched view on Dev across the same four metrics. With no co-tenants, a time-series view is the right lens: each configuration is a single attributable reference run, drawn as one bold curve with no smoothing, so the jaggedness is the real burst structure of its submissions sampled at 300 s, not noise to be filtered. Original drives the main-scheduler and submit-RPC rates substantially above either array-enabled configuration, in line with its per-pipeline submission rate in Fig. 3. The polling-RPC panel (d) shows the same ordering and the largest absolute gap, confirming on a clean cluster that the status-polling load `process.array` eliminates is the dominant controller cost the array refactor removes, agreeing with the distributional view on Phoenix (Fig. 4, panel d).

E. Refactor over patch; study limits

Across both axes the comparison supports the paper’s main strategic finding: the full nf-core refactor is fastest on walltime and brings structural benefits no in-place patch can deliver (testability, container reproducibility, parameter cleanliness, recovery semantics), at a total submission count modestly above the single-process patch but still orders of magnitude below the unmodified upstream pipeline.

We measure on a single site (ASU), with one workflow (`make_lastz_chains`) and three target–query sample pairs (all below the 4 GB threshold, so all runs took the V0 .2bit path). The Phoenix blocked-concurrent design inflates absolute walltimes by self-contention, so for Phoenix we report configuration ordering and within-block ratios rather than absolute values; the matched Dev runs supply the absolute-value reference under co-tenancy-free attribution. The flip in `process.array`’s walltime benefit between the two clusters (Section V-C) rests on a single cluster of each kind and is not claimed to generalize. The findings are specific to Slurm; LSF and PBS sites would need separate measurement. Replication on additional clusters and workloads is planned (Section VI).

VI. DISCUSSION

nf-core defaults are cloud-native. P6 surfaces a recurring mismatch: adapting nf-core’s standard `withLabel` tiers to a Fairshare-aware HPC site requires an explicit overlay (backfill-fit sizing, job arrays, partition routing, resource caps, controller-rate mitigations). HPC sites considering nf-core adoption should budget time for this overlay rather than expecting the defaults to fit.

Cluster architecture for fan-out workflows. The Fairshare → backfill chain that motivates P6 has a provisioning corollary: for high-fan-out Nextflow workflows, a cluster of many older nodes can outperform one of fewer faster nodes, because backfill capacity scales with the number of independent gap-filling slots rather than per-slot speed. Our cross-cluster evaluation supports this [3]: Phoenix’s larger-node-count `public` partition delivered higher LASTZ-stage throughput than the newer-hardware Dev cluster despite per-node CPU disadvantages. Recycling older hardware into a high-node-count cluster, as ASU has done with Phoenix, is the

right shape of compute for nf-core fan-out workloads, with the Slurm tuning of Section II-C its controller-side counterpart.

Future work. Two near-term directions. First, a comparative evaluation of orchestration strategies for Nextflow on shared HPC, broadening the configuration axis beyond the three measured here. Second, `nf-core lint` compliance toward upstream contribution to the nf-core registry: the canonical manifest, CI scaffolding, `conf/` split, per-module `environment.yml` and `meta.yml`, and standard profile set that the refactor has not yet adopted. Replication on additional sites, workloads, and non-Slurm schedulers extends the study limits in Section V-E.

VII. CONCLUSION

The framework for modern scientific workflows is mature; what the literature under-serves is everything around it on a shared HPC cluster. Two gaps carried this refactor: the HPC-aware overlay that a Slurm Fairshare cluster demands over nf-core’s cloud-native defaults, and the verification discipline that keeps a migration honest when much of the code is machine-generated. Both widen, not shrink, as AI assistants take over more of the typing and contribute failures that are quiet by construction. The refactored pipeline, its container, and the parity tooling are released at https://github.com/NilaBlueshirt/make_lastz_chains/tree/nf-core-refactor, with benchmark scripts and data at <https://github.com/NilaBlueshirt/WMS-on-HPC> and the upstream pull request at [21]. We offer the ten practices and the three-part discipline as a portable starting point for the long tail of pre-nf-core tools, hoping to advance the sustainability of scientific software.

ACKNOWLEDGMENTS

We thank the Earth BioGenome Project for the initial opportunity to work on `make_lastz_chains` and for continued collaboration, the ASU Research Computing team for cluster access and operational support, and the upstream `make_lastz_chains` authors at the Hiller Lab for the original pipeline implementation and continued collaboration.

AI use disclosure.: Claude (Anthropic) was used as a coding assistant during the pipeline refactor described in this paper, as discussed in Sections III and IV, and was also used to assist in drafting the manuscript. All technical claims, evaluation results, and final wording are the authors’ responsibility.

REFERENCES

- [1] B. M. Kirilenko, C. Munegowda, E. Osipova, D. Jebb, V. Sharma, M. Blumer, A. K. Morales, A. M. Ahmed, D.-V. Kontopoulos, L. Hilgers, K. Lindblad-Toh, E. K. Karlsson, Zoonomia Consortium, and M. Hiller, “Integrating gene annotation with orthology inference at scale,” *Science*, vol. 380, no. 6643, 2023.
- [2] N. T. Mu, W. Dizon, G. Otero, and T. Battelle, “Optimizing Nextflow-based software on shared HPC resources: A case study with `make_lastz_chains`,” in *US Research Software Engineering Conference 2025 (USRSE’25)*, Philadelphia, PA, USA, Sep. 2025. doi:10.5281/zenodo.17118383.
- [3] N. T. Mu, W. Dizon, G. Otero, and T. Battelle, “Tuning workflow management systems for shared HPC resources,” presented at the *Rocky Mountain Advanced Computing Consortium HPC Symposium (RMACC)*, Boise, ID, USA, May 2026. doi:10.5281/zenodo.20192500.

- [4] P. A. Ewels, A. Peltzer, S. Fillinger, H. Patel, J. Alneberg, A. Wilm, M. U. Garcia, P. Di Tommaso, and S. Nahnsen, “The nf-core framework for community-curated bioinformatics pipelines,” *Nature Biotechnology*, vol. 38, no. 3, pp. 276–278, 2020.
- [5] B. E. Langer *et al.*, “Empowering bioinformatics communities with Nextflow and nf-core,” *Genome Biology*, 2025. doi:10.1186/s13059-025-03673-9.
- [6] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, “Nextflow enables reproducible computational workflows,” *Nature Biotechnology*, vol. 35, pp. 316–319, 2017. doi:10.1038/nbt.3820.
- [7] W. J. Kent, R. Baertsch, A. Hinrichs, W. Miller, and D. Haussler, “Evolution’s cauldron: duplication, deletion, and rearrangement in the mouse and human genomes,” *Proc. Natl. Acad. Sci.*, vol. 100, no. 20, pp. 11484–11489, 2003.
- [8] R. S. Harris, *Improved Pairwise Alignment of Genomic DNA*, Ph.D. thesis, The Pennsylvania State University, 2007.
- [9] J. Yalim, “Toward dynamically controlling Slurm’s classic Fairshare algorithm,” in *Practice and Experience in Advanced Research Computing 2020: Catch the Wave (PEARC ’20)*, ACM, 2020, pp. 538–542. doi:10.1145/3311790.3399628.
- [10] M. Feathers, *Working Effectively with Legacy Code*, Prentice Hall, 2004.
- [11] K. Beck, “Augmented coding: Beyond the vibes,” *Tidy First Substack*, June 2025. <https://tidyfirst.substack.com/p/augmented-coding-beyond-the-vibes>.
- [12] Anthropic, “How Anthropic teams use Claude Code,” 2025. <https://www.anthropic.com/news/how-anthropic-teams-use-claude-code>.
- [13] J. Becker *et al.*, “Measuring the impact of early-2025 AI on experienced open-source developer productivity,” arXiv:2507.09089, 2025.
- [14] Ziyao Zhang *et al.*, “LLM hallucinations in practical code generation: phenomena, mechanism, and mitigation,” *Proceedings of the ACM on Software Engineering (Proc. ACM SE)*, 2024. doi:10.1145/3728894.
- [15] Nextflow Project, “array directive — Nextflow process reference,” accessed May 2026. <https://docs.seqera.io/nextflow/reference/process>.
- [16] Seqera Labs, “Nextflow on BIG IRON: Twelve tips for improving the effectiveness of pipelines on HPC clusters,” accessed May 2026. <https://seqera.io/blog/best-practices-deploying-pipelines-with-hpc-workload-managers/>.
- [17] N. T. Mu, “make_lastz_chains_onPhx: deployment recipe for the original pipeline on the ASU Phoenix cluster,” GitHub repository, accessed May 2026. https://github.com/NilaBlueshirt/make_lastz_chains_onPhx.
- [18] Arizona State University Research Computing, “Sol Supercomputer documentation,” accessed May 2026. <https://asurc.atlassian.net/wiki/spaces/RC/>.
- [19] D. M. Jennewein *et al.*, “The Sol Supercomputer at Arizona State University,” in *Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good (PEARC ’23)*, ACM, 2023, pp. 296–301. doi:10.1145/3569951.3597573.
- [20] Earth BioGenome Project contributors, “Issue #56: make_lastz_chains faToTwoBit index overflow on >4 GB genomes (and downstream LASTZ failure on -long .2bit output),” GitHub, hillierlab/make_lastz_chains, accessed May 2026. https://github.com/hillierlab/make_lastz_chains/issues/56.
- [21] N. T. Mu, “Refactor make_lastz_chains to nf-core DSL2 (pull request #109): chain-level parity comparison against upstream,” GitHub, hillierlab/make_lastz_chains, accessed May 2026. https://github.com/hillierlab/make_lastz_chains/pull/109#issuecomment-4481509551.